

Nodejs

Theory

1. File System (fs)

- Provides methods to interact with files.
- Common methods:
 - `writeFileSync()` → create/write file.
 - `appendFileSync()` → append data.
 - `readFileSync()` → read file.
 - `unlinkSync()` → delete file.
- Useful for logging, configuration, and persistence.

2. Path Module (path)

- Helps work with file and directory paths.
- Methods:
 - `join()` → joins paths.
 - `basename()` → filename.
 - `dirname()` → directory name.
 - `extname()` → file extension.

3. OS Module (os)

- Provides system-related information.
- Examples:
 - `os.hostname()` → system hostname.
 - `os.platform()` → OS platform.
 - `os.totalmem()` / `os.freemem()` → memory info.

4. EventEmitter

- Core Node.js module for handling events.
- Methods:
 - `on(event, listener)` → register listener.
 - `emit(event)` → trigger event.
 - `once(event, listener)` → register one-time listener.
- Enables decoupled, event-driven programming.

5. Buffers

- Used to handle binary data in Node.js.
- Methods:
 - `Buffer.alloc(size)` → allocate buffer.
 - `Buffer.from(string)` → create buffer from string.
 - `toString()` → convert buffer to string.
- Essential for file I/O, streams, and networking.

6. HTTP Server

- `http.createServer()` → creates a basic web server.
- Handles requests (`req.url`) and sends responses (`res.end()`).
- `server.listen(port)` → starts server.
- Forms the foundation of web applications.

Coding Round Questions

- File system → Write a program to create, append, read, and delete a file.
- Path module → Demonstrate `basename`, `dirname`, `join`, and `dirname`.
- OS module → Print system memory and hostname.
- EventEmitter → Create a custom event and trigger it multiple times.
- Buffer → Write a program to create a buffer, write data, and convert it to string.

- HTTP server → Create a server with routes /home and /about.

⚡ Machine Coding Round Challenge

Problem: Logging & Monitoring System

Requirements:

- Create a logger.js module with logInfo() and systemInfo().
- Log messages to logs.txt using fs.
- Use os to log system info.
- Use EventEmitter to trigger events like "start", "heartbeat", "stop".
- Use Buffers to store temporary logs before writing.
- Create an HTTP server with routes:
 - /logs → display logs.
 - /status → display system info.

Logger.js

```
const fs = require("fs");
const path = require("path");
const os = require("os");

const logFile = path.join(__dirname, "logs.txt");

function logInfo(message) {
  console.log(message);
  fs.appendFileSync(logFile, `${message} \n`);
}

function systemInfo() {
  console.log("systeminfo");

  fs.appendFileSync(
    logFile,
    `System : ${os.hostname} | platform: ${os.platform}\n`,
  );
}

module.exports = { logInfo, systemInfo };
```

heartbeat.js

```
const { systemInfo, logInfo } = require("./logger");

systemInfo();

logInfo("Application started");

const id = setInterval(() => {
  logInfo("Heartbeat");
}, 5000);

setTimeout(() => {
  clearInterval(id);
  logInfo("Application stopped");
}, 30000);
```

```
const server = http.createServer( (req,res)=>{

  if(req.url == "/about"){
    console.log(req.url);

    return res.end("About casdfdsfasf");

  }else if(req.url == "/home"){
    return res.end("This is home page");
  }else{
    return res.end("NO match routes found");
  }
})

server.listen(3000, ()=>{
  console.log("Server is up and running");
})
```

```
// event emitter
const EventEmitter = require("events");

const events = new EventEmitter();

// register
```

```
events.on("click", function(){
    console.log("User is clicked", );
})

events.emit("click" );

events.emit("click" );

events.once("salary", function(){
    console.log("salary credited for this month");
})

events.emit("salary")

events.emit("salary");

let buffer = Buffer.alloc(50);

buffer.write("Hello world");

console.log(buffer);
console.log(buffer.toString());

let buffer1 = Buffer.from("Hello FIIT academy");

console.log(buffer1.length);

console.log(buffer1.toString());
```